Step 3 One-Pager — CFR Variants and Monte Carlo Methods

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

April 2026

Problem. Step 2's vanilla CFR visits every information set on every iteration, which stops being affordable one game up from Kuhn. The field's two answers are a better-constant full-traversal method (**CFR+**) and *sampling* the tree (**MCCFR**), and the received wisdom is that sampling is what made real poker solvable. Step 3 asks which one actually wins, on what size of game, and why — the question that decides the solver every later step depends on.

Approach. All four algorithms hand-coded in Python + NumPy on **Leduc Poker** (6 cards, two betting rounds, a revealed community card, 936 information sets, 10,200 nodes, 120 deals): vanilla CFR, CFR+ (regret flooring, linear averaging, alternating updates), MCCFR **external sampling** (all traverser actions, one sampled opponent action), and MCCFR **outcome sampling** (one root-to-terminal trajectory, `eps = 0.6` on-policy mixture with importance-sampling correction). Graded by an exact information-set-constrained best response, under a common **180-second wall-clock budget**, with OpenSpiel as the reference solver. **All numbers below are measured.**

Key results (measured).

- *CFR+ is the workhorse, and it is not close.* At essentially the same iteration count as vanilla CFR (**3,706 vs 3,713**), CFR+ reaches **2.6e-5** exploitability against vanilla's **4.4e-3** — roughly **170x** better, from three localised code changes.
- *MCCFR loses badly at this game size.* External sampling reaches **5.5e-2** after **3.5M** iterations and outcome sampling **1.0e-1** after **8.3M** — three to four orders of magnitude worse than full traversal despite ~1,000x more iterations.
- *The reason is variance, not speed, and it is quantified.* Sampling is **945x / 2,228x** cheaper per iteration but needs ~**99,000x / ~520,000x** more of them. Netted into a wall-clock constant `C_w = C/sqrt(speed)`: **0.075** (vanilla) against **0.767** and **1.143** — sampling is **10.2x / 15.3x** slower to any given accuracy.
- *The crossover is derivable and Leduc is far below it.* Sampling breaks even at roughly **2.1M nodes** (external) and **4.8M** (outcome); Leduc's 10,200 nodes are **210x / 466x** too small, while limit Hold'em (~1e14) is orders of magnitude past it. This reconciles "MCCFR was essential for real poker" with "MCCFR loses here" — same algorithm, opposite side of the line.
- *Cross-validated against OpenSpiel* at a matched 500-iteration budget: vanilla **0.020 vs 0.022**, CFR+ **8.6e-4 vs 9.4e-4**.
- *One measured anomaly, unexplained.* Vanilla CFR beats its own worst-case rate: the supposedly constant `eps*sqrt(T)` falls from **0.90** (T=100) to **0.27** (T=3,700) instead of holding flat, so the `O(1/sqrt(T))` bound is loose here in a way the bound itself does not predict.

Thesis connection. This step fixes the tooling for everything that follows: Leduc plus the exact best-response evaluator become the standard testbed of Steps 7-12, and CFR+ becomes the Nash reference against which every later opponent-modelling, safe-exploitation and LLM number is bracketed. The exactness-versus-variance trade-off measured here returns in Step 5, where the sampled estimates feed a network instead of a table.

Open questions. Whether the crossover node count is a property of the algorithms or of this implementation's constant factors — a vectorised or compiled full-traversal solver shifts `speed_v` and moves the threshold, so the number is honest for this code and not yet a claim about the algorithms in general. Which sampling scheme survives once the strategy is a neural network rather than a table (Step 5). And why vanilla CFR converges faster than its worst case.